Have you ever been watching Formula 1 and been bored because the driver sitting on pole wins the race week after week? Have you ever been on the edge of your seat because someone who started at the back clawed their way to the front lap after lap?

Of course you have!

This analysis seeks to answer the age-old question: can final position be predicated based on starting position? This is a univariate analysis focusing on a small sample of races during Ferrari's Schumacher hay-day.

```
import warnings
warnings.filterwarnings('ignore')  # Suppress all warnings

warnings.warn("This warning will be hidden")
print("Script continues...")
```

```
Script continues...
```

```
#Step 1: upload the data
#I'm using Google Colab, so the CSV files goes in directly into files -> content

#import packages
import pandas as pd
import matplotlib.pyplot as mat

f1_data = pd.read_csv('F1 Data set - Starts and Stops.csv', encoding='latin1')
f1_data.head()
```

	Pos	No	Driver	Car	Laps	Time	Pts	Year	Location	Race_ID	Starting_Positi
0	1	1	ÊHakkinen	McLaren Mercedes	NaN	01:30.6	NaN	2000	Australia	2000_Australia_ÊHakkinen	1
1	2	2	ÊCoulthard	McLaren Mercedes	NaN	01:30.9	NaN	2000	Australia	2000_Australia_ÊCoulthard	2
2	3	3	ÊSchumacher	Ferrari	NaN	01:31.1	NaN	2000	Australia	2000_Australia_ÊSchumacher	3
3	4	4	ÊBarrichello	Ferrari	NaN	01:31.1	NaN	2000	Australia	2000_Australia_ÊBarrichello	4
4	5	5	ÊFrentzen	Jordan Mugen Honda	NaN	01:31.4	NaN	2000	Australia	2000_Australia_ÊFrentzen	5

Next steps: Generate code with f1_data | View recommended plots | New interactive sheet

I built my own data set for this analysis using publicly available data on the F1 website: https://www.formula1.com/en/results/1950/races/94/great-britain/starting-grid.

I made two tables in Excel, one of starting grids, one of race results, since F1 does not combine these tables already. I then assigned each row a unique ID that allowed me to match up the starting position and result for each driver in each race. This analysis treats each individual driver's performance in the race as an individual data point / row.

```
#the encoding added a character at the beginning of some strings, so let's remove that!

f1_data['Driver'] = f1_data['Driver'].str.strip('Ê')

chars_to_remove = "Ê"
translation_table = str.maketrans(dict.fromkeys(chars_to_remove))

```

```
f1_data['Race_ID'] = f1_data['Race_ID'].str.translate(translation_table)
f1_data.head()
```

	Pos	No	Driver	Car	Laps	Time	Pts	Year	Location	Race_ID	Starting_Position
0	1	1	Hakkinen	McLaren Mercedes	NaN	01:30.6	NaN	2000	Australia	2000_Australia_Hakkinen	1.0
1	2	2	Coulthard	McLaren Mercedes	NaN	01:30.9	NaN	2000	Australia	2000_Australia_Coulthard	2.0
2	3	3	Schumacher	Ferrari	NaN	01:31.1	NaN	2000	Australia	2000_Australia_Schumacher	3.0
3	4	4	Barrichello	Ferrari	NaN	01:31.1	NaN	2000	Australia	2000_Australia_Barrichello	4.0
4	5	5	Frentzen	Jordan Mugen Honda	NaN	01:31.4	NaN	2000	Australia	2000_Australia_Frentzen	5.0

Next steps: Generate code with f1_data | View recommended plots | New interactive sheet

```
#checking out the data set data types

datatypes = f1_data.dtypes
print(datatypes)
```

```
Pos                 object
No                   int64
Driver              object
Car                 object
Laps               float64
Time                object
Pts                float64
Year                 int64
Location            object
Race_ID             object
Starting_Position  float64
dtype: object
```

```
#dropping NA's and instances where drivers didn't complete the race.
#These instances will mess up the analysis because you cannot predict a crash or mechanical malfunction based on

indexf1 = f1_data[(f1_data['Pos'] == 'NC')].index
f1_data.drop(indexf1, inplace=True)

f1_data = f1_data.dropna()
```

```
print(f1_data)
```

```
      Pos  No      Driver                Car  Laps      Time   Pts  Year  \
22      1   3  Schumacher            Ferrari  71.0   31:35.3  10.0  2000
23      2  11  Fisichella  Benetton Playlife  71.0  +39.898s   6.0  2000
24      3   5    Frentzen  Jordan Mugen Honda  71.0  +42.268s   4.0  2000
25      4   6      Trulli  Jordan Mugen Honda  71.0  +72.780s   3.0  2000
26      5   9  Schumacher       Williams BMW  70.0    +1 lap   2.0  2000
...    ..  ..         ...                ...   ...       ...   ...   ...
1076    7   7    Heidfeld    Sauber Petronas  52.0    +1 lap   0.0  2002
1077    8  24        Salo             Toyota  52.0    +1 lap   0.0  2002
1078    9  16      Irvine    Jaguar Cosworth  52.0    +1 lap   0.0  2002
1079   10  23      Webber   Minardi Asiatech  51.0   +2 laps   0.0  2002
1080   11   5  Schumacher       Williams BMW  48.0       DNF   0.0  2002

      Location                Race_ID  Starting_Position
22      Brazil  2000_Brazil_Schumacher                3.0
23      Brazil  2000_Brazil_Fisichella                5.0
24      Brazil    2000_Brazil_Frentzen                7.0
```

```
25     Brazil      2000_Brazil_Trulli              12.0
26     Brazil  2000_Brazil_Schumacher               3.0
...       ...                     ...               ...
1076    Japan     2002_Japan_Heidfeld              12.0
1077    Japan         2002_Japan_Salo              13.0
1078    Japan       2002_Japan_Irvine              14.0
1079    Japan       2002_Japan_Webber              18.0
1080    Japan   2002_Japan_Schumacher               1.0

[635 rows x 11 columns]
```

```
#switching the key columns to integers so it's easier to call them later

f1_data['Pos'] = f1_data['Pos'].astype(int)
f1_data['No'] = f1_data['No'].astype(int)
f1_data['Starting_Position'] = f1_data['Starting_Position'].astype(int)
```

```
#switching Year to be a float so it isn't a continuous variable in graphs
f1_data['Year'] = f1_data['Year'].astype(float)
```

```
display(f1_data.dtypes)
```

	0
Pos	int64
No	int64
Driver	object
Car	object
Laps	float64
Time	object
Pts	float64
Year	float64
Location	object
Race_ID	object
Starting_Position	int64

dtype: object

Now that the data is relatively clean, let's get a better sense of it.

```
#shape of the data

rows = f1_data.shape[0]
cols = f1_data.shape[1]

print("Rows: " + str(rows))
print("Columns: " + str(cols))
```

```
Rows: 635
Columns: 11
```

```
#column names

```

```
for col in f1_data.columns:
    print(col)
```

```
Pos
No
Driver
Car
Laps
Time
Pts
Year
Location
Race_ID
Starting_Position
```

```
#This looks at how many times each individual driver appears in the data set.

f1_data['Driver'].value_counts()
```

Driver	count
Schumacher	77
Coulthard	38
Barrichello	36
Button	35
Villeneuve	31
Heidfeld	29
Fisichella	28
Trulli	25
Irvine	23
Frentzen	23
Hakkinen	22
Salo	21
de la Rosa	21
Alesi	21
Montoya	19
Verstappen	18
Panis	18
R□ikk□nen	17
Mazzacane	12
Wurz	10
Webber	10
Sato	10
Burti	9
Gene	9
Diniz	9
Zonta	9
Bernoldi	9
Alonso	9
McNish	8
Massa	8
Herbert	7
Marques	6
Yoong	6
Enge	2

dtype: int64

Schumacher was in 102 races, while Enge was only in 2. Most drivers were in 50 races. Schumacher's overrepresentation in the data set is a concern, but since we are not taking driver into account, only starting position, I am not going to make any changes.

```
#There were 17 different locations for races.

f1_data['Location'].value_counts()
```

Location	count
Great Britain	45
Spain	45
Hungary	44
France	43
Europe	43
USA	43
Japan	43
Belgium	42
Canada	42
Malaysia	40
Italy	38
Austria	35
Brazil	34
Germany	31
San Marino	23
Australia	22
Monaco	22

dtype: int64

```
#there were 20 participating car companies

f1_data['Car'].value_counts()
```

Car	count
Ferrari	79
McLaren Mercedes	67
Williams BMW	64
Sauber Petronas	63
BAR Honda	57
Jaguar Cosworth	49
Jordan Honda	37
Prost Acer	23
Benetton Renault	23
Minardi Fondmetal	20
Renault	20
Benetton Playlife	19
Toyota	19
Arrows Asiatech	19
Jordan Mugen Honda	16
Minardi European	16
Minardi Asiatech	15
Arrows Supertec	12
Prost Peugeot	9
Arrows Cosworth	8

dtype: int64

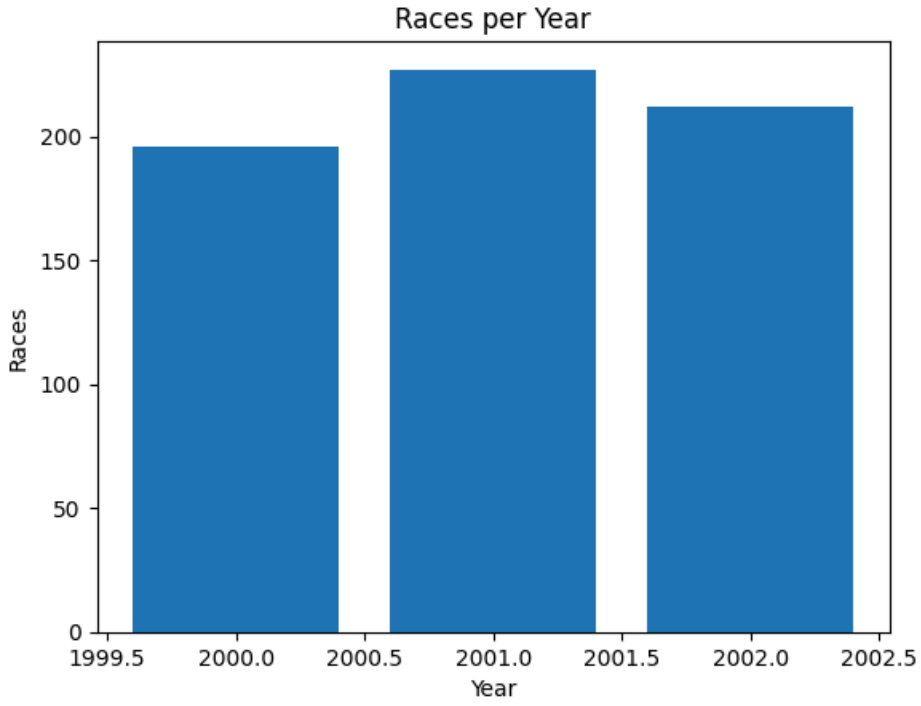
```
#The data covers races across three years: 2000, 2001, 2002

f1_data['Year'].value_counts()
```

Year	count
2001.0	227
2002.0	212
2000.0	196

dtype: int64

```
#there are roughly the same number of races per year

years_tab = f1_data['Year'].value_counts()

mat.bar(years_tab.index, years_tab.values)
mat.title('Races per Year')
mat.xlabel('Year')
```

```
mat.ylabel('Races')
mat.show()
```




```
#subset data frame to have just winners

winners = f1_data[f1_data["Pos"] == 1]
winners.head()
```

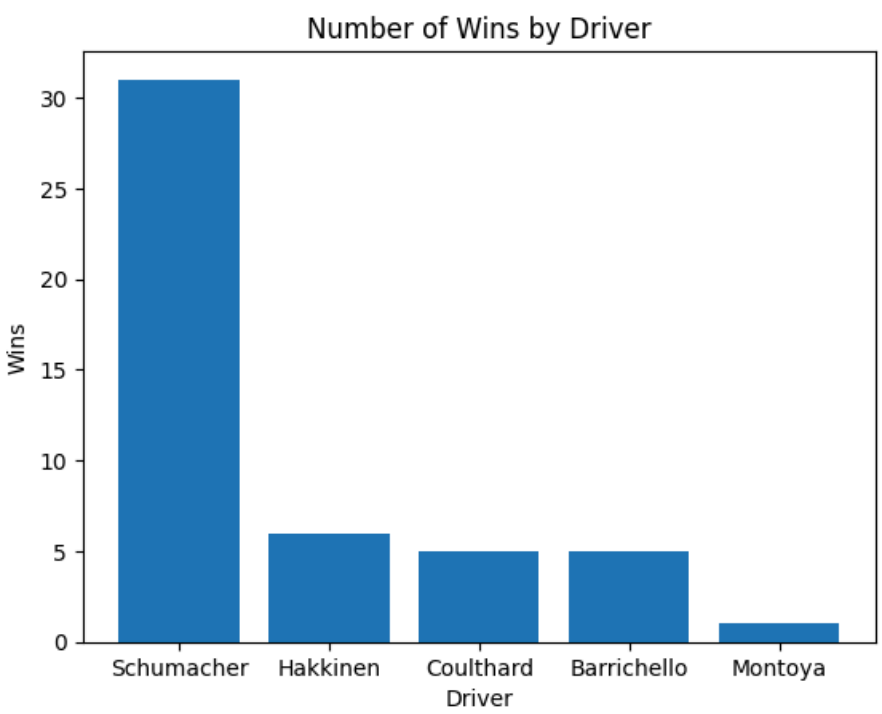
	Pos	No	Driver	Car	Laps	Time	Pts	Year	Location	Race_ID	Starting_Positi
22	1	3	Schumacher	Ferrari	71.0	31:35.3	10.0	2000.0	Brazil	2000_Brazil_Schumacher	
63	1	2	Coulthard	McLaren Mercedes	60.0	28:50.1	10.0	2000.0	Great Britain	2000_Great Britain_Coulthard	
85	1	1	Hakkinen	McLaren Mercedes	65.0	33:55.4	10.0	2000.0	Spain	2000_Spain_Hakkinen	
107	1	3	Schumacher	Ferrari	67.0	42:00.3	10.0	2000.0	Europe	2000_Europe_Schumacher	
149	1	3	Schumacher	Ferrari	69.0	41:12.3	10.0	2000.0	Canada	2000_Canada_Schumacher	

Next steps: Generate code with `winners` View recommended plots New interactive sheet

```
#Common Winners

winners.head()

table_winners = pd.crosstab(winners['Driver'], 'no_of_times')
print(table_winners)
```

```
col_0        no_of_times
Driver
Barrichello            5
Coulthard              5
Hakkinen               6
Montoya                1
Schumacher            31
```

```
winners_tab = winners['Driver'].value_counts()

mat.bar(winners_tab.index, winners_tab.values)
mat.title('Number of Wins by Driver')
mat.xlabel('Driver')
mat.ylabel('Wins')
mat.show()
```



To no one's surprise, Shumacher won almost every race. In case you were wondering, he is the GOAT. At least in this analysis. Next, Hakkinen, Coulthard, and Barrichello won roughly the same number of races in the 3-year span the data set uses. Montoya finishes out the winner's circle. So, over the course of 3 years, only 5 drivers managed to win a race at all.

```
#subset data frame to have just drivers who started on pole

good_start = f1_data[f1_data["Starting_Position"] == 1]
good_start.head()
```

	Pos	No	Driver	Car	Laps	Time	Pts	Year	Location	Race_ID	Starting_Posit
88	4	9	Schumacher	Williams BMW	65.0	+37.311s	3.0	2000.0	Spain	2000_Spain_Schumacher	
89	5	3	Schumacher	Ferrari	65.0	+47.983s	2.0	2000.0	Spain	2000_Spain_Schumacher	
109	3	2	Coulthard	McLaren Mercedes	66.0	+1 lap	4.0	2000.0	Europe	2000_Europe_Coulthard	
149	1	3	Schumacher	Ferrari	69.0	41:12.3	10.0	2000.0	Canada	2000_Canada_Schumacher	
162	14	9	Schumacher	Williams BMW	64.0	DNF	0.0	2000.0	Canada	2000_Canada_Schumacher	

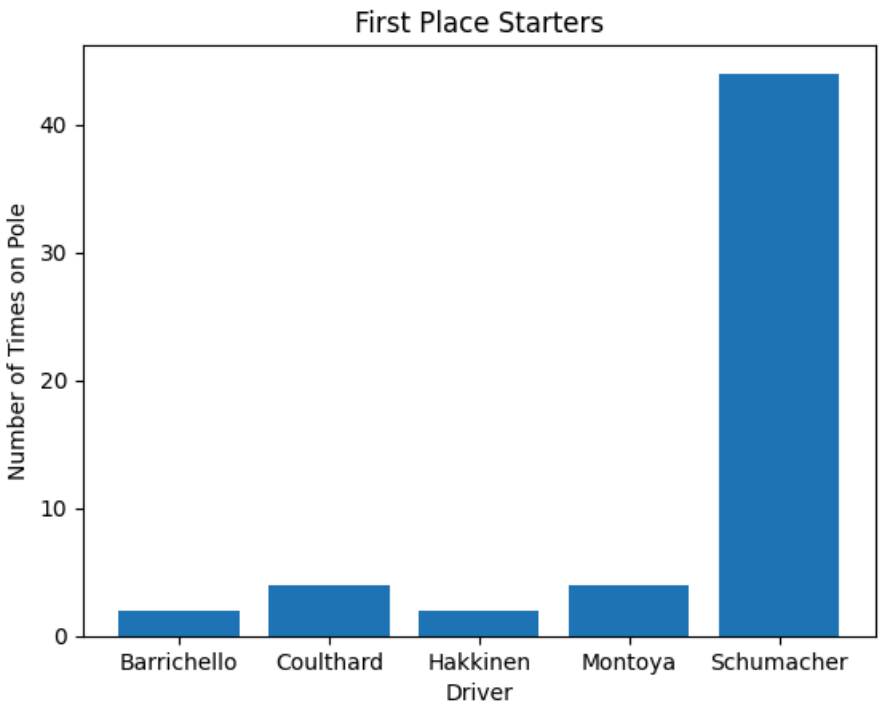
Next steps: Generate code with good_start | View recommended plots | New interactive sheet

```
#Common 1st Place Starters

```

```
3 first_place_starters = pd.crosstab(good_start['Driver'], 'no_of_times')
4 print(first_place_starters)
```

```
col_0        no_of_times
Driver
Barrichello            2
Coulthard              4
Hakkinen               2
Montoya                4
Schumacher            44
```

```
1 mat.bar(first_place_starters.index, first_place_starters['no_of_times'].values)
2 mat.title('First Place Starters')
3 mat.xlabel('Driver')
4 mat.ylabel('Number of Times on Pole')
5 mat.show()
```



Unsurprisingly, and in support of this anlysis' hypothesis, Schumacher also started on pole more often than anyone else. Also, notice that the drivers who started on pole are the same ones who won the races. One interesting note is that Montoya managed to win fewer races than the he started on pole.

```
1 #Winning Car Companies
2
3 table_companies = pd.crosstab(winners['Car'], 'no_of_times')
4 print(table_companies)
```

```
col_0            no_of_times
Car
Ferrari                   32
McLaren Mercedes          11
Williams BMW               5
```

Only 3 car companies managed to produce winning drivers. On the surface, this is not wholely surprising; better cars will make it easier to win, and better drivers will have an easier time winning. And, then, of course, the best drivers will be recruited by the best car companies or vise versa.Thus, it stands to reason that these aspects are highly intertwined.

```
#Wins by Driver and Year

winners.value_counts(['Driver','Car', 'Year'])
```

			count
Driver	**Car**	**Year**	
Schumacher	**Ferrari**	**2002.0**	11
		2001.0	9
		2000.0	7
Hakkinen	**McLaren Mercedes**	**2000.0**	4
Barrichello	**Ferrari**	**2002.0**	4
Schumacher	**Williams BMW**	**2001.0**	3
Hakkinen	**McLaren Mercedes**	**2001.0**	2
Coulthard	**McLaren Mercedes**	**2000.0**	2
		2001.0	2
		2002.0	1
Barrichello	**Ferrari**	**2000.0**	1
Montoya	**Williams BMW**	**2001.0**	1
Schumacher	**Williams BMW**	**2002.0**	1

dtype: int64

It is interesting that Schumacher somehow managed to drive for both Ferrari and Williams in the same year. That's sort of wacky. Also a horribly bad oopsie by whoever at Ferrari allowed that to happen. But their loss is Williams' gain!

```
#Wins by Driver and Country

winners.value_counts(['Location', 'Driver'])
```

Location	Driver	count
Japan	**Schumacher**	3
Canada	**Schumacher**	3
Malaysia	**Schumacher**	3
Australia	**Schumacher**	2
San Marino	**Schumacher**	2
Belgium	**Schumacher**	2
France	**Schumacher**	2
Germany	**Schumacher**	2
Brazil	**Schumacher**	2
Europe	**Schumacher**	2
Spain	**Schumacher**	2
Austria	**Coulthard**	1
	Schumacher	1
France	**Coulthard**	1
Europe	**Barrichello**	1
Brazil	**Coulthard**	1
Belgium	**Hakkinen**	1
Austria	**Hakkinen**	1
Great Britain	**Schumacher**	1
	Hakkinen	1
	Coulthard	1
Germany	**Barrichello**	1
Italy	**Barrichello**	1
Hungary	**Schumacher**	1
	Barrichello	1
	Hakkinen	1
Italy	**Montoya**	1
	Schumacher	1
Monaco	**Schumacher**	1
	Coulthard	1
Spain	**Hakkinen**	1
USA	**Barrichello**	1
	Hakkinen	1
	Schumacher	1

dtype: int64

I was interested to see if there was a pattern to which driver won on which circut, but given Schumacher's dominance, it's hard to tell.

```
#Winners Who Started in 1st

pole_winner  = winners[winners["Starting_Position"] == 1]
pole_winner.value_counts(["Starting_Position", 'Driver'])
```

		count
Starting_Position	**Driver**	
1	**Schumacher**	19
	Hakkinen	2
	Barrichello	1
	Montoya	1

dtype: int64

This table shows instances where the drivers who started on pole also won. Conspicuously, Coulthard is missing; he won despite not starting on pole. If we remember, Schumacher started on pole 44 times, and won 31 races. But, he only won 19 of the races he started on pole (61%). Which means a certain percentage of the time, he is starting on pole and not finising first, and similarly, he is sometimes winning despite not starting in first.

Next, we can look at instances where drivers won without starting in first place. Rogue winners, if you will.

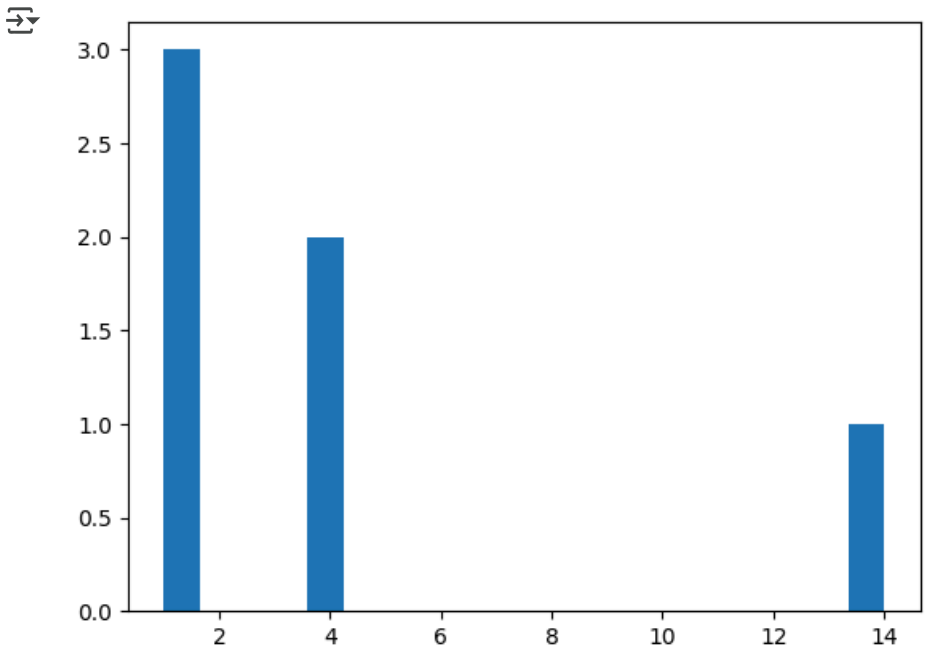
```
#Winners Who Didn't Start in 1st

rogue_winner  = winners[winners["Starting_Position"] != 1]
rogue_winner.value_counts(['Driver', "Starting_Position"])
```

		count
Driver	**Starting_Position**	
Schumacher	**2**	9
	3	3
Barrichello	**4**	2
Coulthard	**2**	2
Hakkinen	**2**	2
Barrichello	**2**	1
	18	1
Coulthard	**4**	1
	7	1
	5	1
Hakkinen	**4**	1
	3	1

dtype: int64

The drivers who are able to finish first despite not starting in first are *still* the usual front runners. And, based on this table, most of the time they start 2nd or 3rd in the line up. Though, there are instances, like Barrichello's start in 18th, where someone is able to climb up from a poor starting position, though these are rare.

```
#rogue winners, the graph

rogue_hist = rogue_winner.value_counts(["Starting_Position"])

mat.hist(rogue_hist, bins = 20)
mat.show()
```



Next, let's take a closer look at some of the trends we've been seeing by isolating specific drivers.

```
#Creationg the Schumacher & Hakkinen data sets

schumacher = f1_data[f1_data["No"] == 3]
hakkinen = f1_data[f1_data["No"] == 1]
```

```
schumacher.head()
```

	Pos	No	Driver	Car	Laps	Time	Pts	Year	Location	Race_ID	Starting_Positio
22	1	3	Schumacher	Ferrari	71.0	31:35.3	10.0	2000.0	Brazil	2000_Brazil_Schumacher	
65	3	3	Schumacher	Ferrari	60.0	+19.917s	4.0	2000.0	Great Britain	2000_Great Britain_Schumacher	
89	5	3	Schumacher	Ferrari	65.0	+47.983s	2.0	2000.0	Spain	2000_Spain_Schumacher	
107	1	3	Schumacher	Ferrari	67.0	42:00.3	10.0	2000.0	Europe	2000_Europe_Schumacher	
149	1	3	Schumacher	Ferrari	69.0	41:12.3	10.0	2000.0	Canada	2000_Canada_Schumacher	

Next steps: Generate code with schumacher | View recommended plots | New interactive sheet

```
hakkinen.head()
```

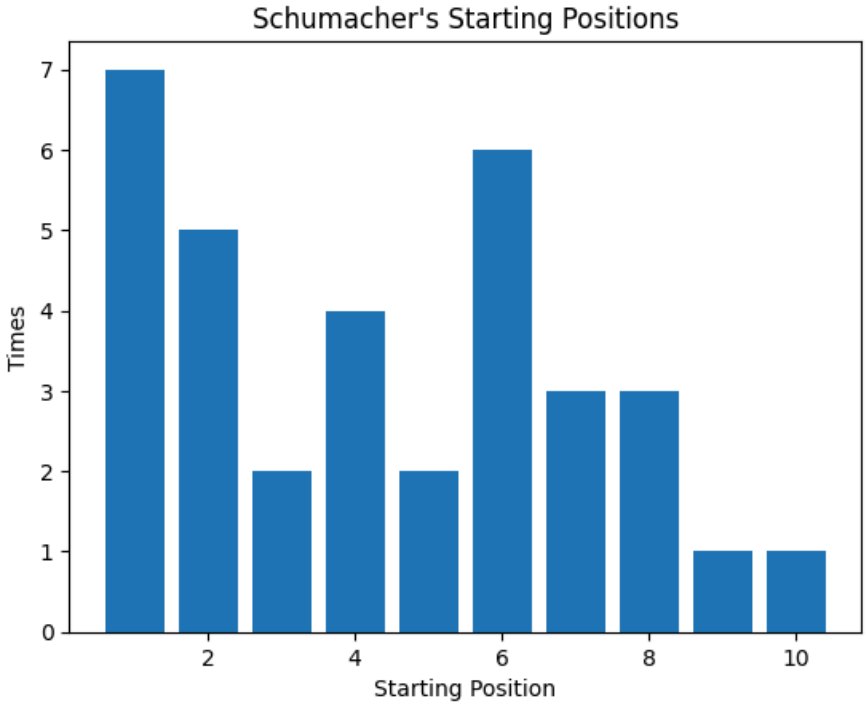
	Pos	No	Driver	Car	Laps	Time	Pts	Year	Location	Race_ID	Starting_Position
64	2	1	Hakkinen	McLaren Mercedes	60.0	+1.477s	6.0	2000.0	Great Britain	2000_Great Britain_Hakkinen	3
85	1	1	Hakkinen	McLaren Mercedes	65.0	33:55.4	10.0	2000.0	Spain	2000_Spain_Hakkinen	2
108	2	1	Hakkinen	McLaren Mercedes	67.0	+13.821s	6.0	2000.0	Europe	2000_Europe_Hakkinen	3
152	4	1	Hakkinen	McLaren Mercedes	69.0	+18.561s	3.0	2000.0	Canada	2000_Canada_Hakkinen	4
172	2	1	Hakkinen	McLaren Mercedes	72.0	+14.748s	6.0	2000.0	France	2000_France_Hakkinen	4

Next steps: Generate code with hakkinen | View recommended plots | New interactive sheet

This next section takes a closer look at our lead drivers and the instances in which they won vs started in 1st, or not. We'll start with Schumacher.

```
#Schumacher Starting Positions

table_schumacher = pd.crosstab(schumacher['Starting_Position'], 'no_of_times')
print(table_schumacher)

mat.bar(table_schumacher.index, table_schumacher['no_of_times'])
mat.xlabel('Starting Position')
mat.ylabel('Times')
mat.title("Schumacher's Starting Positions")
mat.show()
```

```
col_0              no_of_times
Starting_Position
1                            7
2                            5
3                            2
4                            4
5                            2
6                            6
7                            3
8                            3
9                            1
10                           1
```

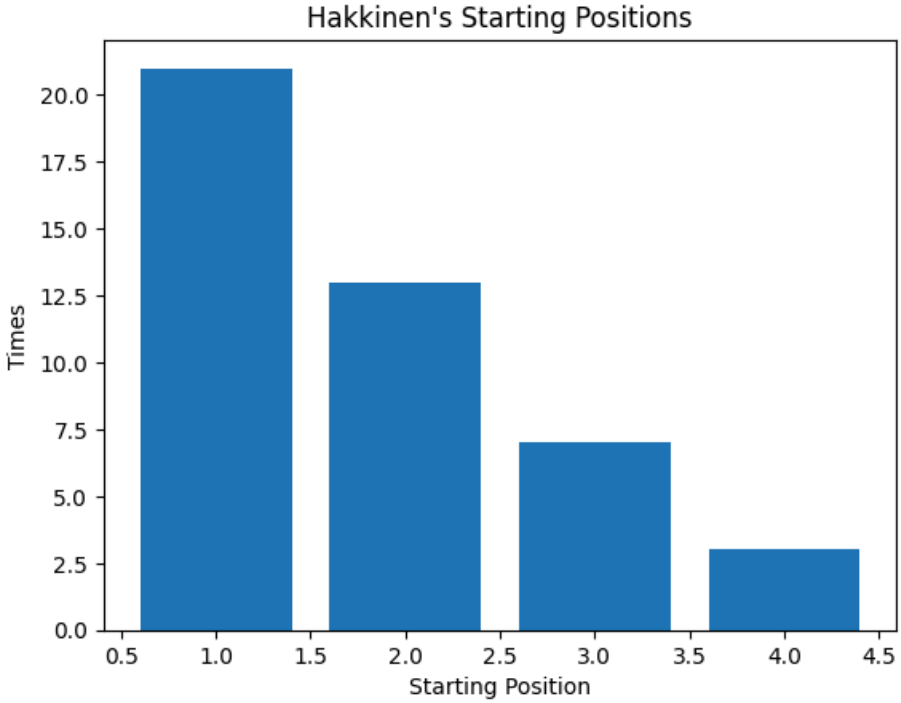


Schumacher only ever started in the top ten, and started in 1st more frequently than any other position. However, he did have some instances of starting in 6th.

Now we can look at Hakkinen.

```
#Hakkinen Starting Positions

table_hakkinen = pd.crosstab(hakkinen['Starting_Position'], 'no_of_times')
print(table_hakkinen)

mat.bar(table_hakkinen.index, table_hakkinen['no_of_times'])
mat.xlabel('Starting Position')
mat.ylabel('Times')
mat.title("Hakkinen's Starting Positions")
mat.show()
```

```
col_0              no_of_times
Starting_Position
1                           21
2                           13
3                            7
4                            3
```



Hakkinen actually started in 1st more often than Schumacher. But, as we saw, he usually doesn't finish in first despite this. And he never started out of the top 5.

```
#For races Schumacher didn't win, what position did he finish in, and which did he start in

schumacher_lose = schumacher[schumacher["Pos"] != 1]
schumacher_lose.head()
```

	Pos	No	Driver	Car	Laps	Time	Pts	Year	Location	Race_ID	Starting_Posit
65	3	3	Schumacher	Ferrari	60.0	+19.917s	4.0	2000.0	Great Britain	2000_Great Britain_Schumacher	
89	5	3	Schumacher	Ferrari	65.0	+47.983s	2.0	2000.0	Spain	2000_Spain_Schumacher	
238	2	3	Schumacher	Ferrari	77.0	+7.916s	6.0	2000.0	Hungary	2000_Hungary_Schumacher	
260	2	3	Schumacher	Ferrari	44.0	+1.103s	6.0	2000.0	Belgium	2000_Belgium_Schumacher	
396	6	3	Hakkinen	McLaren Mercedes	55.0	+48.606s	1.0	2001.0	Malaysia	2001_Malaysia_Hakkinen	

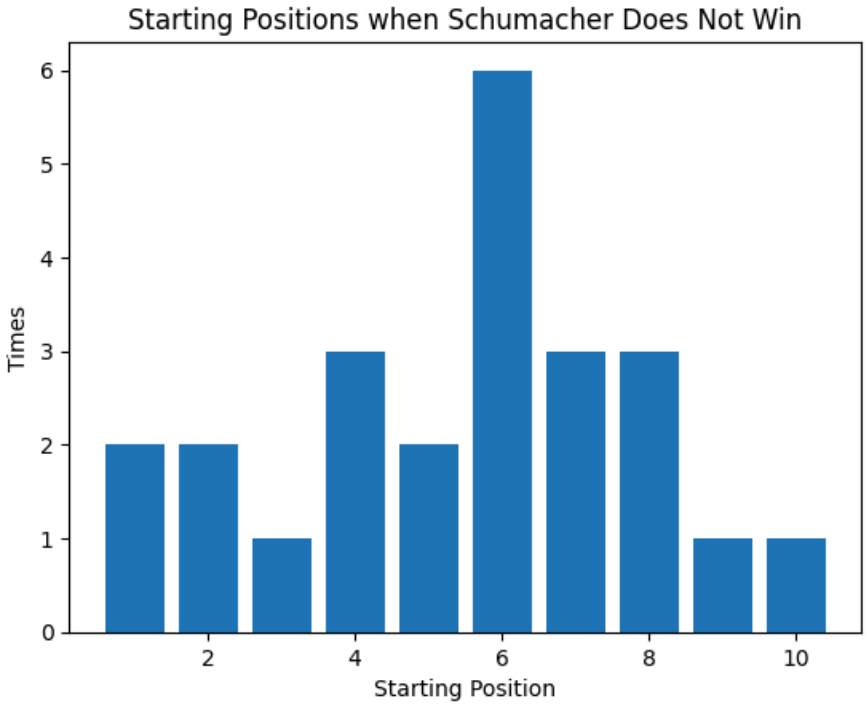
Next steps: Generate code with schumacher_lose | View recommended plots | New interactive sheet

The next logical question is: If he didn't win, where did he end up? Based on the table above, somewhere in teh top 10, so not quite so far from where he started.

We'll look at this again for Schumacher, too.

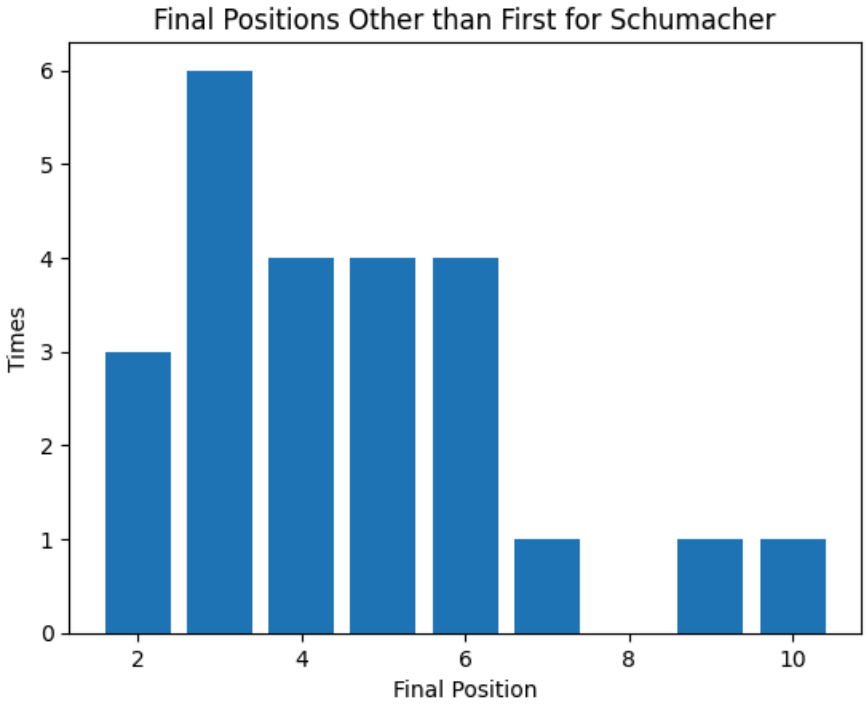
```
table_schumacher_lose = pd.crosstab(schumacher_lose['Starting_Position'], 'no_of_times')
print(table_schumacher_lose)

mat.bar(table_schumacher_lose.index, table_schumacher_lose['no_of_times'])
mat.xlabel('Starting Position')
mat.ylabel('Times')
mat.title('Starting Positions when Schumacher Does Not Win')
mat.show()
```

```
col_0              no_of_times
Starting_Position
1                            2
2                            2
3                            1
4                            3
5                            2
6                            6
7                            3
8                            3
9                            1
10                           1
```




```
table_schumacher_lose_2 = pd.crosstab(schumacher_lose['Pos'], 'no_of_times')
print(table_schumacher_lose_2)

mat.bar(table_schumacher_lose_2.index, table_schumacher_lose_2['no_of_times'])
mat.xlabel('Final Position')
mat.ylabel('Times')
mat.title('Final Positions Other than First for Schumacher')
mat.show()
```

```
col_0  no_of_times
Pos
2                3
3                6
4                4
5                4
6                4
7                1
9                1
10               1
```

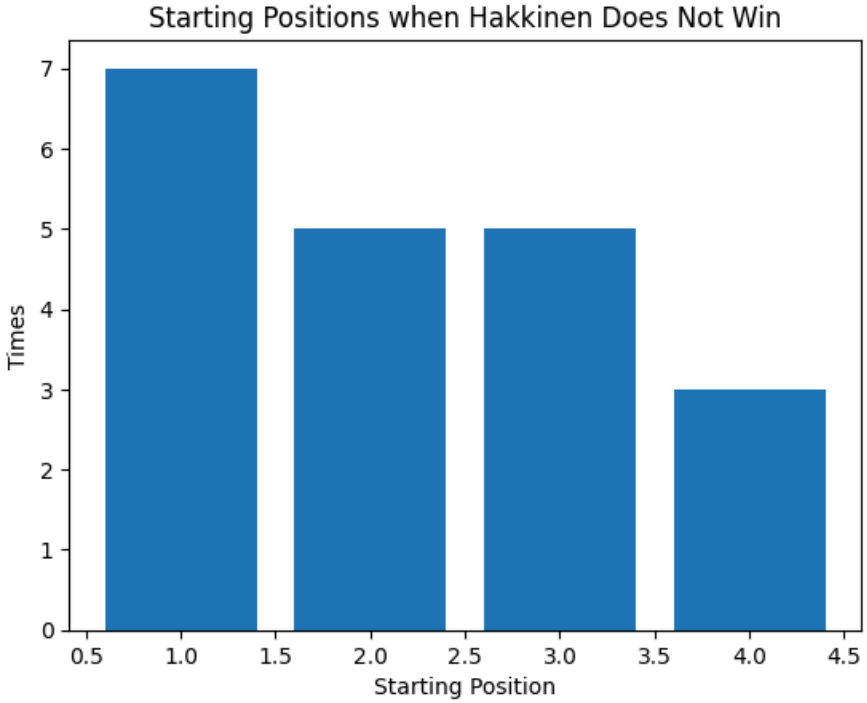


When Schumacher doesn't win, he's actually most likely to start in 6th place. And he is most likely to finish in 3rd.

We can repeat the analysis for Hakkinen.

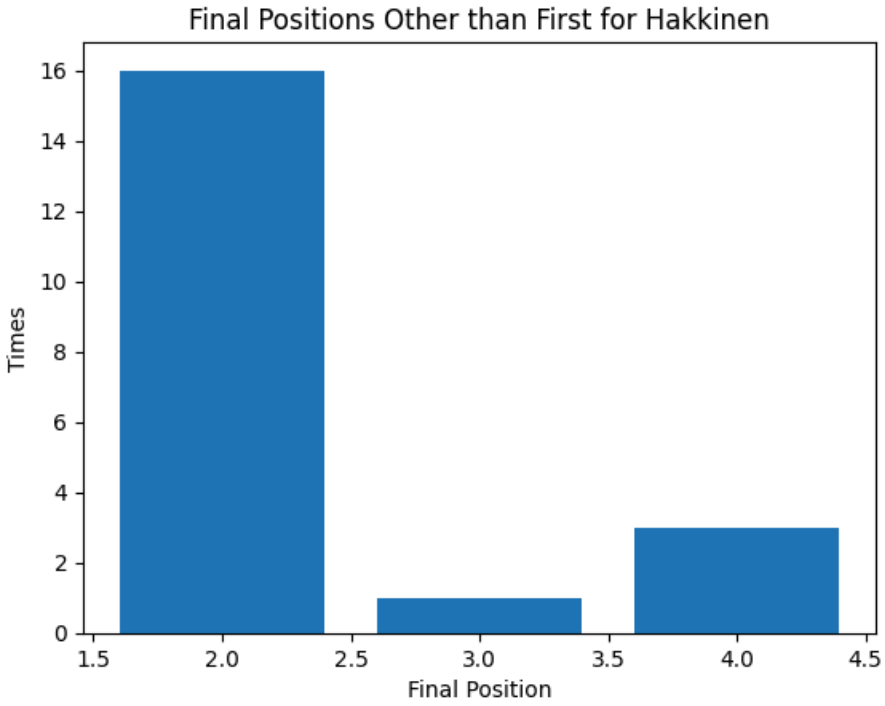
```
hakkinen_lose = hakkinen[hakkinen["Pos"] != 1]
hakkinen_lose.head()

table_hakkinen_lose = pd.crosstab(hakkinen_lose['Starting_Position'], 'no_of_times')
print(table_hakkinen_lose)

mat.bar(table_hakkinen_lose.index, table_hakkinen_lose['no_of_times'])
mat.xlabel('Starting Position')
mat.ylabel('Times')
mat.title('Starting Positions when Hakkinen Does Not Win')
mat.show()
```

```
col_0              no_of_times
Starting_Position
1                            7
2                            5
3                            5
4                            3
```




```
table_hakkinen_lose_2 = pd.crosstab(hakkinen_lose['Pos'], 'no_of_times')
print(table_hakkinen_lose_2)

mat.bar(table_hakkinen_lose_2.index, table_hakkinen_lose_2['no_of_times'])
mat.xlabel('Final Position')
mat.ylabel('Times')
mat.title('Final Positions Other than First for Hakkinen')
mat.show()
```

```
col_0  no_of_times
Pos
2               16
3                1
4                3
```



When Hakkinen doesn't win, he's most likely to have started in 1st, anyway. And he will usually finish 2nd.

This is in line with our analysis. Hakkinen isn't moving around a lot in the order, regardless of 1st place. Schumacher has more variation, but even then, he's staying in the top 10 a lot of the time, so only gaining or losing a few places per race.

We can see if this is an overall trend amongst the drivers.

```
#For all drivers, what position did they finish in, and which did they start in

start_v_stop = f1_data.value_counts(["Pos", "Starting_Position"])
start_v_stop.sort_index(ascending=True)
```

		count
Pos	**Starting_Position**	
1	**1**	23
	2	14
	3	4
	4	4
	5	1
...	**...**	...
17	**2**	1
	6	1
	10	1
	21	1
	22	1

222 rows × 1 columns

dtype: int64

With a final position of 1, the starting position is most often 1, but could also be 2, 3, 4, 5, etc. So, clearly there are some doubts that the pole-sitted will finish that way. However, it is *more likely*.

This next section repeats some of the previous analysis with Montoya.

```
#Montoya - starts on pole, doesn't finish 1st


Montoya = f1_data[f1_data["Driver"] == "Montoya"]

#how many times did he start in each position
table_Montoya = pd.crosstab(Montoya['Starting_Position'], 'no_of_times')
print(table_Montoya)
```

```
col_0              no_of_times
Starting_Position
1                            4
2                            2
3                            1
4                            6
5                            1
6                            2
8                            2
12                           1
```

Montoya usually starts in 6th, but he did start in 1st a few times. However, he didn't finish first very often.

```
Montoya.head()
```

	Pos	No	Driver	Car	Laps	Time	Pts	Year	Location	Race_ID	Starting_Position
458	2	6	Montoya	Williams BMW	65.0	+40.738s	6.0	2001.0	Spain	2001_Spain_Montoya	12
545	2	6	Montoya	Williams BMW	67.0	+4.217s	6.0	2001.0	Europe	2001_Europe_Montoya	3
591	4	6	Montoya	Williams BMW	60.0	+68.772s	3.0	2001.0	Great Britain	2001_Great Britain_Montoya	8
638	8	6	Montoya	Williams BMW	76.0	+1 lap	0.0	2001.0	Hungary	2001_Hungary_Montoya	8
670	1	6	Montoya	Williams BMW	53.0	16:58.5	10.0	2001.0	Italy	2001_Italy_Montoya	1

Next steps: Generate code with Montoya | View recommended plots | New interactive sheet

```
#when Montoya won, where did he start?

Montoya_wins = Montoya[Montoya["Pos"] == 1]
Montoya_wins.head()

```

	Pos	No	Driver	Car	Laps	Time	Pts	Year	Location	Race_ID	Starting_Position
670	1	6	Montoya	Williams BMW	53.0	16:58.5	10.0	2001.0	Italy	2001_Italy_Montoya	1

Montoya only won 1 race, despite starting on pole 4 times. But, he did start in 1st for that race.

```
#when he didn't win, where did he start?

Montoya_lose = Montoya[Montoya["Pos"] != 1]
table_Montoya_lose = pd.crosstab(Montoya_lose['Starting_Position'], 'no_of_times')
print(table_Montoya_lose)
```

```
col_0              no_of_times
Starting_Position
1                            3
2                            2
3                            1
4                            6
5                            1
6                            2
8                            2
12                           1
```

Montoya most often starts in 4th, but he has a wide range of starting positions.

So we're seeing the same sort of trend. The pole sitter wins, and there isn't a whole lot of variation between starting position and final position, though Montoya does have more variation than Hakkinen.

This next portion transitions to seeing if the final position can be predicted based on starting position. From the analysis so far, results are mixed. While first place finishers are most likely to start in 1st, this isn't always the case. And, part of the reason is that the same guy is starting and finishing first every time.

```
#correlation analysis

```

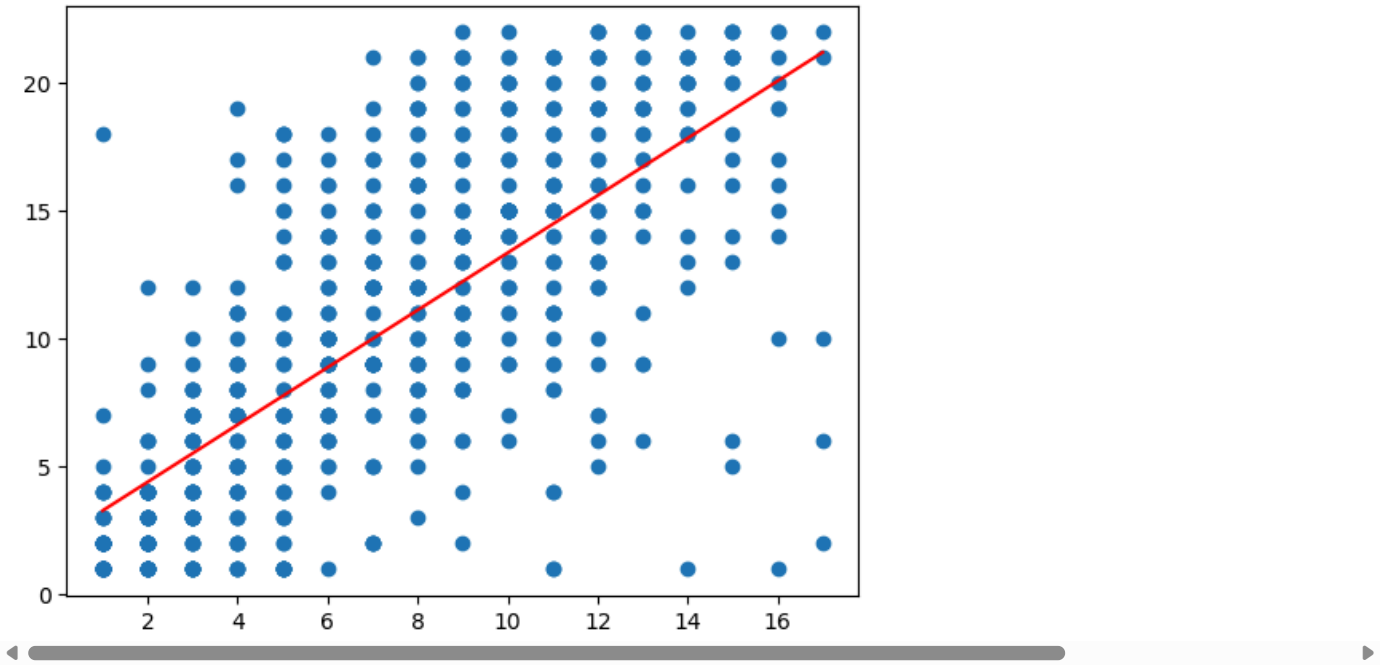
```
f1_data['Pos'] = pd.to_numeric(f1_data['Pos'], errors='coerce')
f1_data['Starting_Position'] = pd.to_numeric(f1_data['Starting_Position'], errors='coerce')

y = pd.Series(f1_data['Starting_Position'])
x = pd.Series(f1_data['Pos'])

correlation = y.corr(x)
correlation
```

```
np.float64(0.7222048164153465)
```

The first step is to determine if starting position and final position are correlated. Spoiler alert: They are; highly correlated.

```
import matplotlib.pyplot as plt
import numpy as np

x = x.dropna()
y = y.dropna()

x = x[np.isfinite(x)]
y = y[np.isfinite(y)]

common_indices = x.index.intersection(y.index)

x = x.reindex(common_indices)
y = y.reindex(common_indices)


plt.scatter(x, y)

plt.plot(np.unique(x), np.poly1d(np.polyfit(x, y, 1))
         (np.unique(x)), color='red')
```

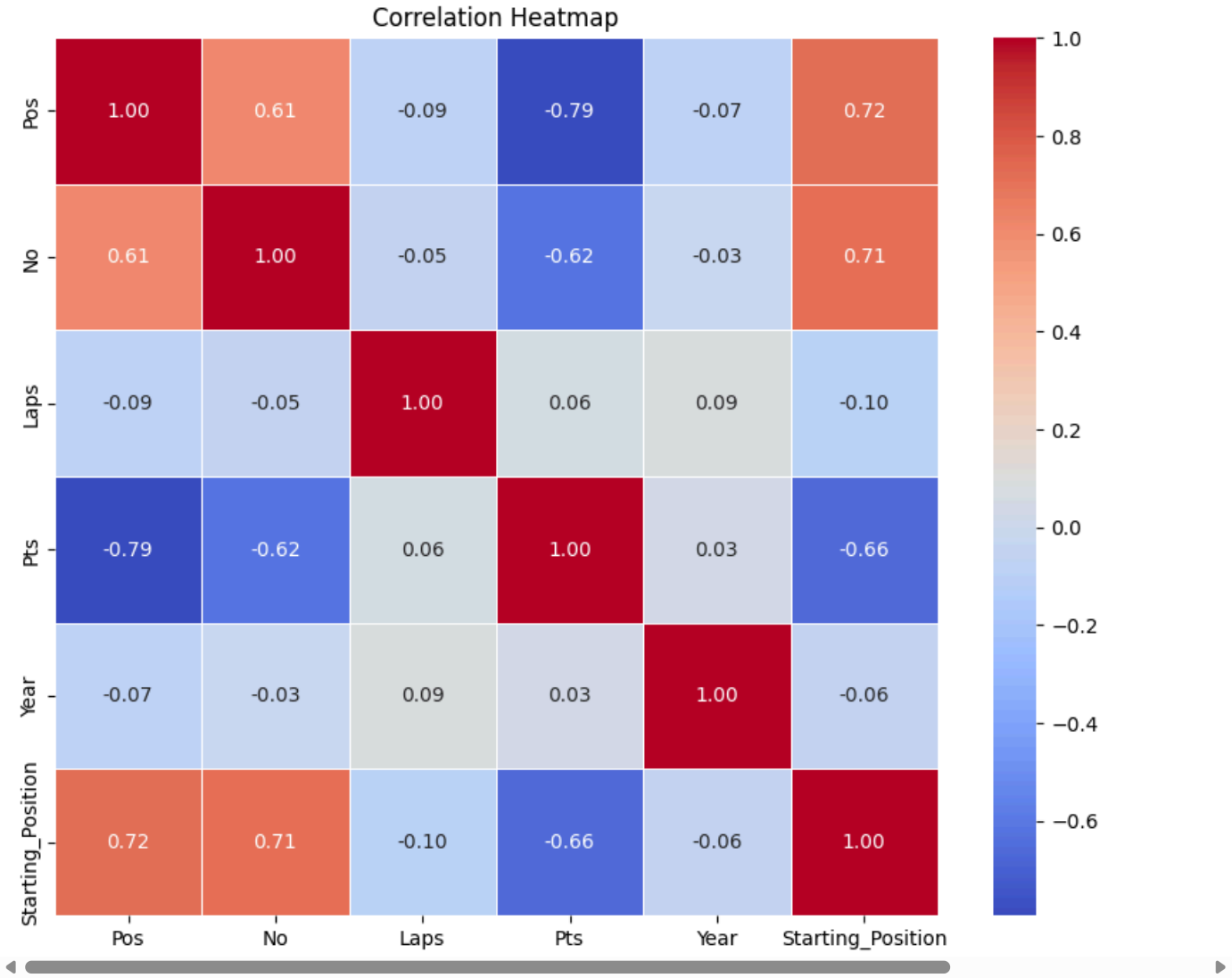
```
[<matplotlib.lines.Line2D at 0x7c7837413550>]
```



That's a pretty convincing positive correlation, yeah?

```
corr_matrix = f1_data.corr(numeric_only=True)

import seaborn as sns
```

```
4
5 # Plot heatmap
6 plt.figure(figsize=(10, 8))
7 sns.heatmap(corr_matrix, annot=True, cmap='coolwarm', fmt=".2f", linewidths=0.5)
8 plt.title('Correlation Heatmap')
9 plt.show()
10
```



However, while starting position and final position are highly correlated, they're not the only thing that is correlated. Final position is also highly correlated with driver (No), and negatively correlated with points (Pts). This isn't unexpected. Schumacher wins everything, so any first place instance is highly likely to be associated with Schumacher.

Next, we can try to model the relationship between starting and final place. First, I will try a Poisson regression. This is an odd choice for this situation, but since Poisson regressions can't be negative, it isn't the worst choice. It will also yield some coefficients, which could be interesting to look at.

```
1 #Poisson Regression
2
3 import statsmodels.api as sm
4 import numpy as np
5 from sklearn.model_selection import train_test_split
6
7 # training and test set
8 train, test = train_test_split(f1_data, test_size=0.2)
```

```
 9
10 # Define dependent variable
11 y_train = train['Pos']
12 y_test = test['Pos']
13
14 # Define independent variables
15 X_train = train[['Starting_Position']]
16 X_test = test[['Starting_Position']]
17
18 # Add intercept term
19 X_train = sm.add_constant(X_train)
20 X_test = sm.add_constant(X_test)
21
22 # --- The change starts here ---
23 # Concatenate X and y for train and test sets
24 train_data = pd.concat([X_train, y_train], axis=1)
25 test_data = pd.concat([X_test, y_test], axis=1)
26
27 # Drop rows with missing values from the combined datasets
28 train_data = train_data.dropna()
29 test_data = test_data.dropna()
30
31 # Separate X and y after dropping missing values
32 X_train = train_data[['const', 'Starting_Position']]  # Include 'const'
33 y_train = train_data['Pos']
34 X_test = test_data[['const', 'Starting_Position']]  # Include 'const'
35 y_test = test_data['Pos']
36
37
38 # Ensure dependent variable is non-negative and an integer
39 y_train = y_train.astype(int) # Convert to integer if not already.
40 y_train = np.where(y_train < 1, 1, y_train) # If y < 1, make it 1 to avoid 0s
41 y_test = y_test.astype(int) # Convert to integer if not already.
42 y_test = np.where(y_test < 1, 1, y_test) # If y < 1, make it 1 to avoid 0s
43
44
45
46 # Fit Poisson regression model
47 poisson_model = sm.GLM(y_train, X_train, family=sm.families.Poisson()).fit()
48
49 # Display model summary
50 print(poisson_model.summary())
51
52 #https://github.com/KenDaupsey/Basic-Poisson-Regression-Using-Python/blob/main/Basic_Poisson_Regression.ipynb
```

```
                 Generalized Linear Model Regression Results
==============================================================================
Dep. Variable:                      y   No. Observations:                  508
Model:                            GLM   Df Residuals:                      506
Model Family:                 Poisson   Df Model:                            1
Link Function:                    Log   Scale:                          1.0000
Method:                          IRLS   Log-Likelihood:                -1262.3
Date:                Sat, 28 Jun 2025   Deviance:                       671.64
Time:                        23:34:16   Pearson chi2:                     721.
No. Iterations:                     4   Pseudo R-squ. (CS):             0.7109
Covariance Type:            nonrobust
===================================================================================
                      coef    std err          z      P>|z|      [0.025      0.975]
-----------------------------------------------------------------------------------
const               1.2382      0.038     32.625      0.000       1.164       1.313
Starting_Position   0.0636      0.003     24.621      0.000       0.059       0.069
===================================================================================
```

From the coefficients, starting position is roughly equal to final position. However, the model isn't significant at all. This is probably because of all the correlations between variables; the assumptions of a Poisson regression aren't being met. Poisson regression requires independence of variables. We can test this more thorougly using the variance.

```
#poisson assumptions - expected vs observed counts
#The mean and variance of the model are equal

# Importing Statistics module
import statistics

# Creating a sample of data
# Use the numerical y_train from the Poisson regression section (which was the 'Pos' column)
# Ensure it is treated as a single-dimensional array or list for statistics functions
sample_data = f1_data['Pos'].dropna()  # Drop missing values
sample_data = sample_data.astype(int)  # Convert to integer
sample_data = np.where(sample_data < 1, 1, sample_data)

print("Variance of sample set is % s"
      %(statistics.variance(sample_data)))

# Also check the mean
print("Mean of sample set is %s"
      %(np.mean(sample_data)))
```

```
Variance of sample set is 17
Mean of sample set is 7.324409448818898
```

After testing the assumptions, Poisson cannot be used as a model. Besides independent variables, the variance and mean must also be equal, and here, it is clear they are not.

But, that is OK! We can try other models, mainly categorical models. The first will be K Nearest Neighbors.

```
# K Nearest Neighbors

from sklearn.neighbors import KNeighborsClassifier
from sklearn.model_selection import train_test_split

# Reshape X to be a 2D array
X = f1_data[['Starting_Position']]  # Use double brackets to select as DataFrame
y = f1_data[['Pos']]

X = X.dropna() #Drop rows with missing values from X
y = y.dropna() #Drop rows with missing values from y

common_indices = X.index.intersection(y.index)
X = X.loc[common_indices]
y = y.loc[common_indices]

X_train, X_test, y_train, y_test = train_test_split(
             X, y, test_size = 0.2, random_state=1)

knn = KNeighborsClassifier(n_neighbors=5)

knn.fit(X_train, y_train)

print(knn.predict(X_test))

#need to get some sort of accuracy, something to compare it to?
#https://www.geeksforgeeks.org/k-nearest-neighbor-algorithm-in-python/ <- in here, scroll down
```

```
[ 1  3  7  7  7 14  3 12  3  9 10  2 12  1  2 14  6 12  2  7 10 14  8  2
 10  9  7  5 14  9  7 10  8  9  3  7  2  8  6 12  7 11 12 11  2  4  3  7
  7  2 10  8 14  2  2  7  3  3 10  2 10  1  2  3 12 10 14 10  4 14  9 14
  5 10  2 14  4  7  7 14  2  1 11  7  9  2  7  4  3  7  3 10 12  6 14  7
  7  7  1 11  2  2  1  6 12  8  1  1  2  2 14  2 10  7 12  3  1  5  6  2
 10  2 10  2  9  2 11]
```

```
#KNN Accuracy
print(knn.score(X_test, y_test))
```

```
0.14173228346456693
```

The accuracy is bad.

```
neighbors = np.arange(1, 9)
train_accuracy = np.empty(len(neighbors))
test_accuracy = np.empty(len(neighbors))

# Loop over K values
for i, k in enumerate(neighbors):
    knn = KNeighborsClassifier(n_neighbors=k)
    knn.fit(X_train, y_train)

    # Compute training and test data accuracy
    train_accuracy[i] = knn.score(X_train, y_train)
    test_accuracy[i] = knn.score(X_test, y_test)

# Generate plot
plt.plot(neighbors, test_accuracy, label = 'Testing dataset Accuracy')
plt.plot(neighbors, train_accuracy, label = 'Training dataset Accuracy')

plt.legend()
plt.xlabel('n_neighbors')
plt.ylabel('Accuracy')
plt.show()
```

We can try to retrain the model to see if we get better results using n=6.

```
#retrain the model

knn_final = KNeighborsClassifier(n_neighbors=6)
knn_final.fit(X_train, y_train)

#predict!

pred_6 = knn_final.predict([[6]])
pred_20 = knn_final.predict([[20]])
pred_1 = knn_final.predict([[1]])

print(pred_6)
print(pred_20)
print(pred_1)

#lol whoops - > it predicts people getting SUPER HIGH places.
```

```
[2]
[14]
[1]
```

K Nearest Neighbors is a better choice of model because it is categorical. However, this model is not any more accurate than the Poisson regression.

When asked to predict, the model did predict that the 1st place starter would finish in 1st, but predicted improvements for starts in 6th and 20th. This isn't really in support of the hypothesis.

```
#naive bayes

from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import accuracy_score

# Initialize the Gaussian Naive Bayes classifier
gnb = GaussianNB()

# Train the model
gnb.fit(X_train, y_train)

# Predict the labels for the test set
y_pred = gnb.predict(X_test)

# Calculate the accuracy
accuracy = accuracy_score(y_test, y_pred)
print(f'Accuracy: {accuracy}')

#https://www.geeksforgeeks.org/ml-naive-bayes-scratch-implementation-using-python/
#https://machinelearningmastery.com/naive-bayes-classifier-scratch-python/
#https://www.geeksforgeeks.org/gaussian-naive-bayes-using-sklearn/
```

```
Accuracy: 0.14960629921259844
```

Lastly, a Naive Bayes model. Again, this is a categorical model, and again, it isn't very accurate.

Now, we're going to switch gears (pun intended). Up until now, we've been trying to predict the ending position, but there's too many factors. The original hypothesis was that drivers couldn't change by too many positions. So! Let's see if we can remove the actual position number and see some improvements.

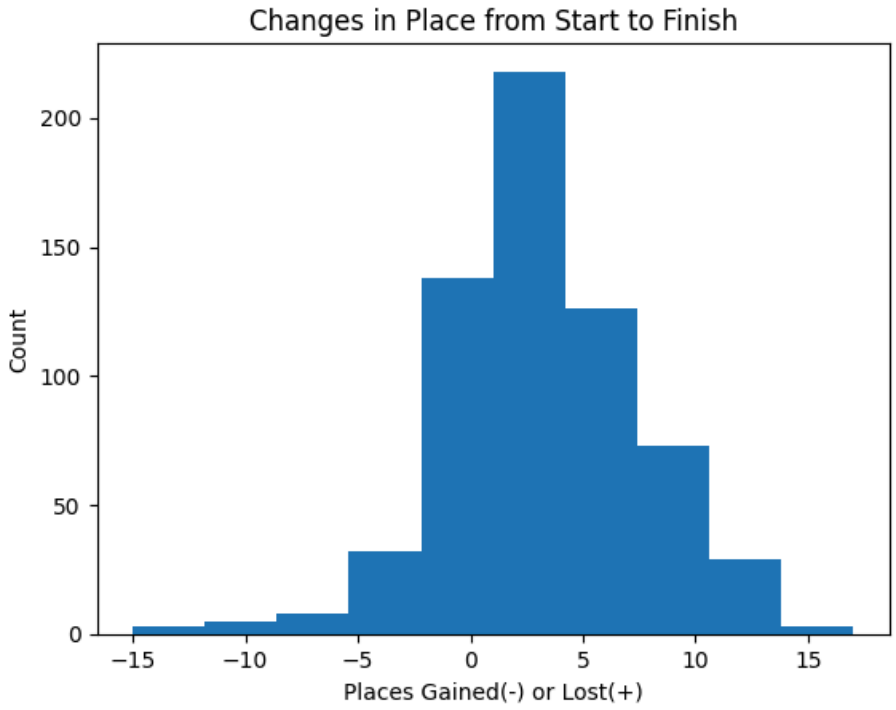
```
#making a new column in the original data
#this one calculates the change in places - it's backwards so it's negative if you move away from 1st,
```

```
#and positive if you move towards 1st

f1_data['delta_place'] = f1_data['Pos'] - f1_data['Starting_Position']
f1_data['delta_place'] = f1_data['delta_place'] * -1
f1_data.head()
```

	Pos	No	Driver	Car	Laps	Time	Pts	Year	Location	Race_ID	Starting_Position
22	1	3	Schumacher	Ferrari	71.0	31:35.3	10.0	2000.0	Brazil	2000_Brazil_Schumacher	3
23	2	11	Fisichella	Benetton Playlife	71.0	+39.898s	6.0	2000.0	Brazil	2000_Brazil_Fisichella	5
24	3	5	Frentzen	Jordan Mugen Honda	71.0	+42.268s	4.0	2000.0	Brazil	2000_Brazil_Frentzen	7
25	4	6	Trulli	Jordan Mugen Honda	71.0	+72.780s	3.0	2000.0	Brazil	2000_Brazil_Trulli	12
26	5	9	Schumacher	Williams BMW	70.0	+1 lap	2.0	2000.0	Brazil	2000_Brazil_Schumacher	3

Next steps: Generate code with f1_data | View recommended plots | New interactive sheet

```
mat.hist(f1_data['delta_place'])
mat.title('Changes in Place from Start to Finish')
mat.xlabel('Places Gained(-) or Lost(+)')
mat.ylabel('Count')
mat.show()
```



Most times, people lose 1-5 places (remember, negative is good). But, a lot will gain 1 or 2 places. It's very rare to either fall to the back of the grid, but it's rarer to claw your way back up once you're there.

```
#added another column - this one is categorical
#lost is bad, gained is good, stayed the same is self explanatory
```

```
3
4 f1_data['place_status'] = f1_data['delta_place'].apply(lambda x: "lost places" if x > 0 else ("gained places" i
5 f1_data.head()
```

	Pos	No	Driver	Car	Laps	Time	Pts	Year	Location	Race_ID	Starting_Positi	n
22	1	3	Schumacher	Ferrari	71.0	31:35.3	10.0	2000.0	Brazil	2000_Brazil_Schumacher		3
23	2	11	Fisichella	Benetton Playlife	71.0	+39.898s	6.0	2000.0	Brazil	2000_Brazil_Fisichella		5
24	3	5	Frentzen	Jordan Mugen Honda	71.0	+42.268s	4.0	2000.0	Brazil	2000_Brazil_Frentzen		7
25	4	6	Trulli	Jordan Mugen Honda	71.0	+72.780s	3.0	2000.0	Brazil	2000_Brazil_Trulli		12
26	5	9	Schumacher	Williams BMW	70.0	+1 lap	2.0	2000.0	Brazil	2000_Brazil_Schumacher		3

Next steps: Generate code with `f1_data` View recommended plots New interactive sheet

We can try to vizualize this.

Lost places = 0 (lightest purple) Gained places = 1 (medium purple) Stayed in Place = 2 (darkest purple)

```
1 import seaborn as sns
2
3 f1_data['kde_hue'] = f1_data['place_status']
4 f1_data['kde_hue'].replace(['lost places', 'gained places', 'stayed in place'],
5                           [0, 1, 2], inplace=True)
6
7
8 sns.kdeplot(data=f1_data, x="Starting_Position", hue="kde_hue", fill = True)
```

```
<Axes: xlabel='Starting_Position', ylabel='Density'>
```